Python 金融数据分析

第 2 章　*Python* 基础语法

主讲人:　王伟冠, 吕文俊
院　系:　经济学院金融系

上海大学

2026 年 10 月 14 日



1. 初识 Python 程序
2. 基本数据类型
3. 运算符与表达式
4. 字符串与格式化输出
5. 基本数据结构
6. 控制流：让程序会” 判断” 与” 重复”
7. 函数：把常用计算” 打包”
8. 类与对象：面向对象入门
9. 综合案例与小结

1. 初识 Python 程序
2. 基本数据类型
3. 运算符与表达式
4. 字符串与格式化输出
5. 基本数据结构
6. 控制流：让程序会” 判断” 与” 重复”
7. 函数：把常用计算” 打包”
8. 类与对象：面向对象入门
9. 综合案例与小结

本章学习目标

- 理解什么是程序、变量与数据类型
- 掌握 Python 的四种基本数据类型：整数、浮点数、布尔、字符串
- 掌握常用运算符与表达式
- 掌握列表、元组、字典、集合四种基本数据结构
- 掌握 `if`、`for`、`while` 控制流与函数定义
- 理解类、对象、属性和方法，能编写简单持仓类
- 能够编写简单的金融计算脚本（收益率、复利等）

对零基础同学的建议

编程像学外语：多敲、多错、多改。看懂不等于会写，请边听课边动手。

程序是什么?

- **程序（Program）**：一系列告诉计算机" 做什么" 的指令，按顺序执行。
- Python 是**解释型语言**：代码由解释器逐行翻译、逐行执行，无需提前编译。
- 两种常见运行方式：
  - **交互模式**：输入一行，立即得到结果（像计算器）；
  - **脚本文件**：把代码保存为 .py 文件，一次执行多行。
- 本课程主要使用 **Jupyter Notebook**（VS Code 中运行）：
  - 代码按" 单元格（cell）" 组织，可逐块运行、即时查看结果；
  - 适合数据分析：代码、结果、文字说明交织在一起。



第一个 Python 程序

```
# 井号开头的是注释：写给人看的，计算机会跳过
print("Hello, Finance!")    # print() 把结果显示出来

# 也可以直接当计算器用
1 + 1
520 * 0.03
```

- print() 是一个**函数**：圆括号里放" 输入"，函数完成某项工作。
- 引号括起来的内容叫**字符串**（文本），不加引号的是**数字**。
- 在 Notebook 中，单元格最后一行的结果会自动显示，无需 print。

易错点

必须使用**英文标点**：引号""、括号 ()、逗号,。中文标点会报错！

变量：给数据起名字

- **变量**是对象的名字：用 = 把名字绑定到右侧对象。
- 之后可以通过名字反复使用、修改这份数据。

```
price = 30.5          # 把 30.5 装进名为 price 的变量
shares = 100
value = price * shares    # 用变量做计算
print(value)          # 输出 3050.0

price = 31.2          # 变量的值可以随时更新
```

注意

= 是**赋值**（把右边的值放进左边），不是数学里的"相等"。

变量命名规则

- 只能包含字母、数字和下划线 `_`，且**不能以数字开头**。
- 区分大小写：`Price` 与 `price` 是两个不同变量。
- 不能使用 Python 保留字：`if`、`for`、`class`、`return` 等。

合法	非法	原因
`stock_price`	`2stock`	不能以数字开头
`rate2026`	`stock price`	不能含空格
`_temp`	`stock-price`	减号会被当作减法

好习惯

变量名要"见名知意"：用 `annual_rate` 而不是 `a`；多个单词用下划线连接。

1. 初识 Python 程序

2. 基本数据类型

3. 运算符与表达式

4. 字符串与格式化输出

5. 基本数据结构

6. 控制流：让程序会" 判断" 与" 重复"

7. 函数：把常用计算" 打包"

8. 类与对象：面向对象入门

9. 综合案例与小结

为什么要关心数据类型?

- 不同类型的数据，能做的运算不同：数字可以相乘，文本不能。
- 金融数据中常见的类型：
  - 股价、利率 → 小数（**浮点数 float**）
  - 成交股数、年份 → **整数 int**
  - 股票代码、公司名 → **字符串 str**
  - 是否涨停、是否 ST → **布尔 bool**（真/假）

Python 的四种基本数据类型

`int`（整数）、`float`（浮点数）、`str`（字符串）、`bool`（布尔）



```
a = 42          # int: 整数
b = 3.14        # float: 浮点数（小数）
c = 1e6         # 科学计数法，等于 1000000.0

type(a)         # <class 'int'>   type() 查看类型
type(b)         # <class 'float'>
```

- type() 是"验货工具"：随时查看某个数据的类型。
- 整数与浮点数混合运算，结果自动变为浮点数：

```
1 + 2.0         # 3.0
10 / 4          # 2.5   注意：/ 的结果永远是 float
```

浮点数的”小陷阱”

```
0.1 + 0.2              # 0.30000000000000004
123456789.12 * 0.01    # 可能尾数不完全精确
```

- 计算机用二进制存储小数，某些十进制小数无法精确表示。
- 这是**所有编程语言**的共同现象，不是 Bug。
- 金融计算中的应对方式：
  - 比较时允许微小误差：`abs(a - b) < 1e-9`；
  - 需要精确金额时使用 `decimal` 模块（进阶内容）；
  - 显示时控制小数位数（后面学格式化输出）。

```python
# bool: 只有 True / False 两个值，表示"真/假"
is_listed = True
5 > 3        # True，比较的结果就是布尔值

# str: 用单引号或双引号括起来的文本
code = "600519"
name = '贵州茅台'
greet = "It's fine"      # 内部有单引号时外层用双引号
```

- True、False 首字母大写，是关键字而非变量。
- 股票代码、日期常以**字符串**存储（保留前导零，如"000001"）。

类型转换

```python
int("100")      # 100    字符串 -> 整数
float("3.5")    # 3.5    字符串 -> 浮点数
str(600519)     # '600519'  数字 -> 字符串
int(3.99)       # 3      直接截断小数，不是四舍五入
```

- 用类型名当函数即可完成转换：`int()`、`float()`、`str()`。
- 无法转换时会报错：`int("abc")`。

典型易错

`"1" + "2"` 结果是`"12"`（字符串拼接），而 `1 + 2` 才是 3。
从文件读入的数字常常是字符串，先转换再计算！

1. 初识 Python 程序

2. 基本数据类型

3. 运算符与表达式

4. 字符串与格式化输出

5. 基本数据结构

6. 控制流：让程序会" 判断" 与" 重复"

7. 函数：把常用计算" 打包"

8. 类与对象：面向对象入门

9. 综合案例与小结

运算符	含义	示例	结果
`+ - * /`	加减乘除	`7 / 2`	`3.5`
`//`	整除（取商的整数部分）	`7 // 2`	`3`
`%`	取余数	`7 % 2`	`1`
`**`	幂运算	`2 ** 10`	`1024`

金融味道：复利终值

本金 100 元、年利率 5%、复利 10 年：

`100 * (1 + 0.05) ** 10` $\approx$ 162.89 元

比较运算符（结果为布尔值）

```
30.5 > 28          # True
price == 30.5      # 是否相等：用两个等号
price != 0         # 不等于
```

逻辑运算符（组合多个条件）

```
ret > 0 and ret < 0.1    # 且
is_st or ret < -0.05     # 或
not is_listed            # 非
```

高频错误

判断相等用 ==；单个 = 是赋值。
"介于两者之间"要写完整：0 < ret < 0.1，不能写成 0 < ret 和 < 0.1。

```python
x = 10
x += 2      # 等价于 x = x + 2
x -= 1      # 等价于 x = x - 1
x *= 1.05   # 等价于 x = x * 1.05（像是"涨了5%"）
```

课堂练习（2 分钟）

某股票昨日收盘价 `close_prev = 20.0`，今日收盘价 `close = 21.5`。
用一行代码计算日收益率 $r = (P_t - P_{t-1})/P_{t-1}$，并打印结果。

```python
ret = (close - close_prev) / close_prev
print(ret)      # 0.075，即上涨 7.5%
```

1. 初识 Python 程序
2. 基本数据类型
3. 运算符与表达式
4. 字符串与格式化输出
5. 基本数据结构
6. 控制流：让程序会” 判断” 与” 重复”
7. 函数：把常用计算” 打包”
8. 类与对象：面向对象入门
9. 综合案例与小结

- 字符串是一串字符的序列，每个字符有编号（**索引**），从 0 开始。

```
code = "600519"
code[0]       # '6'      第1个字符
code[-1]      # '9'      倒数第1个字符
code[0:3]     # '600'    切片：含头不含尾
code[3:]      # '519'
len(code)     # 6        长度
```

为什么从 0 开始?

这是大多数编程语言的约定：索引表示" 距起点多远"，第 1 个字符距离为 0。初学只需记住规则，多敲就习惯了。

```
exchange = "SH" + code       # 拼接: 'SH600519'
"ha" * 3                     # 重复: 'hahaha'

name = "Kweichow Moutai"
name.lower()         # 转为小写
name.upper()         # 转为大写
"2026-08-04".split("-")     # 按分隔符切开 -> ['2026','08','04']
", ".join(["600519", "000858"])  # '600519, 000858'
```

- **方法**：写在对象后面、用点号调用的"功能"，如 `code.upper()`。
- 字符串是**不可变**的：方法返回新字符串，原字符串不变。
- 金融场景中常用 `split()` 拆解日期、代码、文件路径。

f-string：最好用的格式化输出

```python
name = "贵州茅台"
price = 1500.35
ret = 0.0752

print(f"{name} 当前价格 {price} 元")
print(f"日涨幅为 {ret:.2%}")       # 百分比，保留2位小数
print(f"价格保留两位小数: {price:.2f}")
```

- f-string：字符串前加 f，{变量} 会被替换为变量的值。
- 冒号后是格式说明：.2f 保留两位小数，.2% 百分比。
- 写分析报告、打印结果时非常常用，务必熟练。

输出

```
贵州茅台当前价格 1500.35 元
日涨幅为 7.52%
```

1. 初识 Python 程序
2. 基本数据类型
3. 运算符与表达式
4. 字符串与格式化输出
5. **基本数据结构**
6. 控制流：让程序会" 判断" 与" 重复"
7. 函数：把常用计算" 打包"
8. 类与对象：面向对象入门
9. 综合案例与小结

- 如果一只股票有 250 个交易日收盘价，难道要定义 250 个变量？

```
p1 = 30.1; p2 = 30.5; p3 = 29.8; ...   # 不可行！
```

- 我们需要能"一次装下一串数据"的容器——**数据结构**。

结构	写法	可变?	典型用途
列表 list	`[1, 2, 3]`	可变	一串股价、日期
元组 tuple	`(1, 2, 3)`	不可变	固定的一行记录
字典 dict	`{"a": 1}`	可变	代码 → 价格的映射
集合 set	`{1, 2, 3}`	可变	去重、成员判断

列表（list）：最常用的容器

```python
prices = [30.1, 30.5, 29.8, 31.2, 30.9]
codes  = ["600519", "000858", "601318"]
mixed  = [1, "two", 3.0, True]     # 可混装不同类型

prices[0]         # 30.1   索引从0开始
prices[-1]        # 30.9   最后一个
prices[1:3]       # [30.5, 29.8]  切片
len(prices)       # 5
```

- 列表用方括号 `[]` 定义，元素用逗号分隔。
- 索引、切片规则与字符串完全一致（含头不含尾）。
- 列表**可以修改**：`prices[0] = 30.0`。

```python
codes = ["600519", "000858"]
codes.append("601318")       # 末尾追加
codes.insert(0, "000001")    # 在指定位置插入
codes.remove("000858")       # 按值删除
popped = codes.pop()         # 弹出最后一个

prices + [31.0]              # 拼接成新列表
[0] * 5                      # [0, 0, 0, 0, 0]
"600519" in codes            # True, 成员判断
prices.sort()                # 排序（升序）
```

提示

append 是在原列表上修改；而 prices + [31.0] 生成**新**列表，原列表不变。

变量是指向对象的名字。赋值不会自动复制列表；copy() 创建一个新的外层列表。先预测三个列表的内容，再执行。

```
original = [80, 90]
alias = original
copied = original.copy()
alias.append(100)
print(original)          # [80, 90, 100]
print(copied)            # [80, 90]
print(alias is original) # True
```

把 alias[0] 改成 60，哪些列表会改变？如果列表元素本身还是列表，copy() 会复制内层吗？

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

sort() 修改原列表并返回 None；sorted() 返回新列表。不能把 names.sort() 的返回值当作排好序的名单。

```
names = ["Wang", "Li", "Zhang"]
ordered = sorted(names, key=len)
print(names, ordered)
result = names.sort()
print(names)
print(result)  # None
```

元组（tuple）：不可变的列表

```python
record = ("600519", "贵州茅台", 1500.35)
record[0]          # '600519'
record[1] = "茅台"  # 报错！元组元素不可修改
```

- 元组用圆括号（）定义；索引、切片与列表相同。
- **不可变**：一旦创建不能增删改——适合存放"不应被改动"的记录。
- 常见场景：函数返回多个值、一行数据记录。

```python
# 元组解包：一次赋值给多个变量
code, name, price = record
```

- 字典存储**键值对**（key–value），像一本"查得到"的字典。

```
prices = {"600519": 1500.35, "000858": 128.6, "601318": 55.2}
prices["600519"]              # 1500.35，按"键"取值
prices["000001"] = 12.5       # 新增或修改键值对
"601318" in prices            # True

prices.keys()     # 所有键
prices.values()   # 所有值
prices.items()    # 所有(键，值)对
```

金融应用

股票代码 → 收盘价、行业 → 市盈率、指标名 → 计算结果……
凡是"名称 → 数值"的对应关系，优先想到字典。

集合（set）：自动去重

```python
s = {"600519", "000858", "600519"}   # 自动去重
len(s)              # 2

s.add("601318")
"600519" in s       # True，成员判断非常快

a = {"600519", "000858"}
b = {"000858", "601318"}
a & b               # 交集 {'000858'}
a | b               # 并集
```

- 集合**无序、不重复**，用花括号 {} 定义。
- 常用于：数据去重、快速判断某代码是否在某名单中。
- 注意：{} 是空字典；空集合要用 set()。

1. 初识 Python 程序
2. 基本数据类型
3. 运算符与表达式
4. 字符串与格式化输出
5. 基本数据结构
6. 控制流：让程序会” 判断” 与” 重复”
7. 函数：把常用计算” 打包”
8. 类与对象：面向对象入门
9. 综合案例与小结

```python
ret = 0.075

if ret > 0:
    print("今日上涨")
elif ret == 0:
    print("今日平盘")
else:
    print("今日下跌")
```

- 缩进是 Python 的语法：缩进（4 个空格）内的行属于该分支。
- 条件行末尾必须有**冒号** :。
- `elif`、`else` 可省略；分支可嵌套。

零基础同学最大的坑

缩进错误（漏缩进、空格与 Tab 混用）是最常见的报错来源，请使用编辑器自动缩进。

for 循环：遍历序列

```python
codes = ["600519", "000858", "601318"]
for code in codes:              # 依次取出每个元素
    print(code)

for i in range(5):              # range(5) 生成 0,1,2,3,4
    print(i)

for i in range(1, 11):          # 1 到 10（含头不含尾）
    print(i)
```

- for 变量 in 序列：——让循环变量依次等于序列中的每个元素。
- range(起止) 常用来按次数循环；同样注意**含头不含尾**。
- 列表、字符串、字典的键都可以被遍历。

给签到名单编号时，不需要自己维护计数器。enumerate 的 start=1 只改变显示的编号，不改变列表索引。

```
attendees = ["小林", "小周", "小陈"]
for number, person in enumerate(attendees, start=1):
    print(number, person)
```

zip 按位置配对；默认到最短序列就停止。配对前检查长度，避免悄悄漏掉学生。这与 pandas 按标签对齐不同。

```
students = ["小林", "小周", "小陈"]
scores = [82, 95, 68]
print(len(students), len(scores))
for student, score in zip(students, scores):
    print(f"{student}: {score}分")
score_map = dict(zip(students, scores))
print(score_map)
```

输出分数至少为 80 的学生姓名。删去 scores 的最后一个元素，观察配对数量。

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

先理解循环，再使用推导式。读法是“对每个 score，若达到 80，就保留 score”。复杂流程仍分步写。

```
scores = [82, 95, 68]
passed = []
for score in scores:
    if score >= 80:
        passed.append(score)
short_form = [score for score in scores if score >= 80]
print(passed, short_form)
```

例：批量计算日收益率

```python
prices = [100.0, 102.0, 101.0, 105.0]
returns = []                        # 空列表，用于存放结果

for i in range(1, len(prices)):
    r = (prices[i] - prices[i-1]) / prices[i-1]
    returns.append(r)               # 把结果追加进列表

print(returns)    # [0.02, -0.0098..., 0.0396...]
```

- 模式：**准备空列表** → **循环计算** → **append 收集结果**，非常常用。
- `prices[i-1]` 是前一天价格，所以 `i` 从 1 开始。
- （预告：学完 NumPy 后，这段代码可以一行搞定。）

```python
# while: 条件为 True 就一直执行
value = 100
years = 0
while value < 200:          # 复利翻倍需要几年?
    value = value * 1.05
    years = years + 1
print(years)                # 15
```

```python
for i in range(10):
    if i == 3:
        continue            # 跳过本次, 进入下一次
    if i == 6:
        break               # 立即结束整个循环
    print(i)
```

注意

while 循环必须保证条件终会变为 False, 否则程序会"死循环"卡住。

1. 初识 Python 程序
2. 基本数据类型
3. 运算符与表达式
4. 字符串与格式化输出
5. 基本数据结构
6. 控制流：让程序会” 判断” 与” 重复”
7. **函数：把常用计算” 打包”**
8. 类与对象：面向对象入门
9. 综合案例与小结

为什么要用函数?

- 已经用过的"内置函数": `print()`、`len()`、`type()`、`int()`、`max()`、`min()`、`sum()`、`abs()`。
- 函数的好处:
  - **复用**: 写一次，到处调用;
  - **清晰**: 给一段逻辑起个名字，代码像文章一样易读;
  - **易测**: 小段逻辑更容易检查对错。
- 我们可以**自己定义函数**: 把重复使用的计算（如收益率公式）打包。

函数的三要素

名字（做什么）+ **参数**（输入）+ **返回值**（输出）

def 定义函数

```python
def simple_return(p_prev, p_now):
    """计算简单收益率"""
    return (p_now - p_prev) / p_prev

r = simple_return(100, 105)
print(r)              # 0.05
print(f"{r:.2%}")     # 5.00%
```

- def 函数名（参数 1，参数 2):，函数体缩进，return 返回结果。
- 三引号字符串"""..."""是**文档字符串**：说明函数用途。
- 没有 return 的函数返回 None。

默认参数与关键字参数

```python
def future_value(pv, rate, years=1):
    """复利终值: years 默认为 1"""
    return pv * (1 + rate) ** years

future_value(100, 0.05)              # 105.0
future_value(100, 0.05, 10)          # 162.89
future_value(100, rate=0.05, years=3)  # 指明参数名
```

- **默认参数**：years=1，调用时可省略。
- **关键字参数**：写明参数名，顺序随意，可读性好。
- 金融公式（现值、终值、年金）特别适合封装成函数。

课堂练习

编写函数 pv(fv, rate, years) 计算现值 $PV = FV/(1+r)^n$，
并计算：3 年后收到的 100 元，折现率 5%，现值是多少？

不定参数：用 *args 接收多个数

求和时，有时输入两个数，有时输入三个数，能否使用同一个函数？

```python
def total(*args):
    print(args)
    return sum(args)

print(total(10, 20))
# (10, 20)
# 30
print(total(10, 20, 30))
# (10, 20, 30)
# 60
```

- *args 把数量不固定的**位置参数**收集成**元组**。
- args 是惯用名称；也可写 *numbers，函数体相应使用 numbers。
- “不定”指参数数量不固定，同一个函数可处理不同数量的输入。

不定参数：用 **kwargs 接收带名称的信息

学生信息有时只有姓名和成绩，有时还包含城市。

```python
def show_info(**kwargs):
    print(kwargs)

show_info(name="小王", score=90)
# {'name': '小王', 'score': 90}
show_info(name="小李", score=85, city="上海")
# {'name': '小李', 'score': 85, 'city': '上海'}
```

- `**kwargs` 把数量不固定的**关键字参数**收集成**字典**。
- 参数用“名称 = 值”传入；字典的键是参数名，值是传入的信息。
- `kwargs` 是惯用名称，真正起作用的是 `**`。

课堂练习：汇总多笔现金流

编写 `total_cashflow(*amounts)`，求现金流金额之和（元）。
分别传入 `(100, -30)` 和 `(100, -30, 50)`；说明 `amounts` 的类型。参考答案见 Notebook 折叠单元格。

print 负责展示，return 把结果交给调用者。需要继续计算时，应当返回结果。没有 return 的函数调用结果是 None。

```
def show_double(number):
    print(number * 2)

def double(number):
    return number * 2

shown = show_double(3)
returned = double(3)
print(shown, returned + 1)  # None 7
```

输入来自手工登记，存在首尾空格和大小写差异。先清理单个值，再循环处理整个列表；不修改输入列表。

```python
def clean_city(text):
    return text.strip().title()

raw_cities = [" shanghai ", "BEIJING", "shanghai"]
cities = [clean_city(text) for text in raw_cities]
print(cities)
print(sorted(set(cities)))
```

统计每个清理后的城市出现几次，用字典保存。预期 Shanghai 为 2、Beijing 为 1。

检查思路

先写出预期结果，再运行。参考答案见配套 Notebook 的折叠单元格。

从报错最后一行识别类型，再回看出错行。下面错误行保持注释；课堂逐条取消注释观察，观察后再恢复。

```
# float("缺考")        # ValueError: 内容不能转成数值
# "82" + 5            # TypeError: 字符串不能与整数相加
# unknown_score       # NameError: 名字还未定义
# [82, 95][2]         # IndexError: 索引超出范围
print(float("82") + 5)
print([82, 95][1])
```

1. 初识 Python 程序
2. 基本数据类型
3. 运算符与表达式
4. 字符串与格式化输出
5. 基本数据结构
6. 控制流：让程序会" 判断" 与" 重复"
7. 函数：把常用计算" 打包"
8. 类与对象：面向对象入门
9. 综合案例与小结

问题：怎样管理多个矩形？

已知宽和高，怎样求面积？先沿用第 2 章的字典和函数。长度单位为厘米，面积单位为平方厘米。

```python
def rectangle_area(rect):
    return rect["width"] * rect["height"]

rect_a = {"width": 3, "height": 4}
rect_b = {"width": 5, "height": 2}
print(rectangle_area(rect_a), rectangle_area(rect_b))
```

实际输出

```
12 10
```

手算得到 $3 \times 4 = 12$、$5 \times 2 = 10$。字典保存数据，函数接收数据并计算；当相关操作增多时，可以用类组织这些数据和操作。

对应 Notebook：8.1

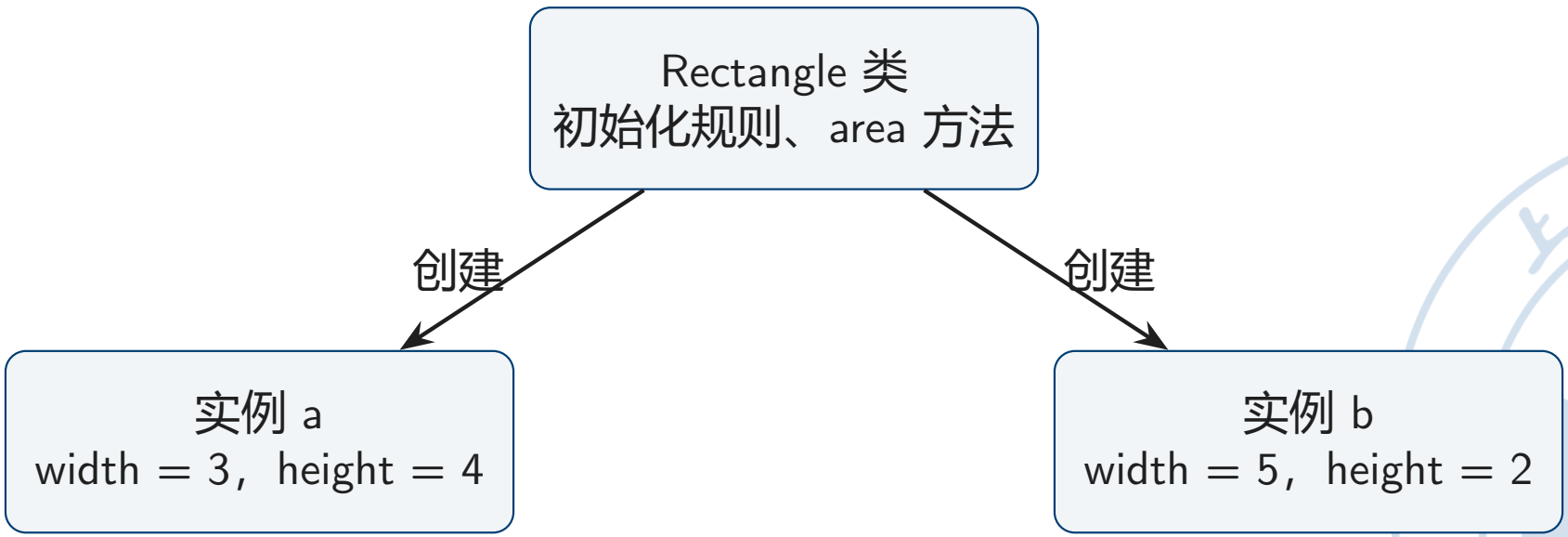
类、对象与实例：先分清三个词

类（class）定义一类对象的属性与行为，例如 Rectangle。对象（object）是程序中具体存在的值；实例强调“这个对象属于哪个类”。

一个具体矩形是 Rectangle 的实例。宽、高是属性（数据），求面积是方法（操作）。同一个类可以创建多个对象，每个对象保存自己的宽和高。

学习目标：读懂“对象. 属性”和“对象. 方法 ()”，写出简单的类，再用股票持仓练习。前面学过的变量、函数、字典和循环，正好是本节的基础。

对应 Notebook：8.2



同一套面积计算规则，分别使用各自的宽与高。
调用 a.area() 得到 12；调用 b.area() 得到 10。

第一个类：矩形的宽、高与面积

class 后面是类名；类体中的 def 定义方法。按缩进看结构：两个方法并列，方法内部再缩进。

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height
```

这里只定义规则，没有创建具体矩形，因此没有输出。__init__ 初始化实例属性；area 读取这些属性并返回面积。

对应 Notebook：8.3

创建两个实例：同一套规则，不同数据

调用 Rectangle(宽, 高) 创建实例。变量 a、b 分别引用两个不同的矩形对象。

```
a = Rectangle(3, 4)
b = Rectangle(5, 2)
print(a.width, a.height, a.area())
print(b.width, b.height, b.area())
print(isinstance(a, Rectangle))
```

实际输出

```
3 4 12
5 2 10
True
```

第一行是 a 的宽、高、面积；第二行是 b 的。isinstance 检查对象是否属于给定类或其子类。类定义通常只写一次，实例可以创建很多个。

对应 Notebook：8.4

self：这一次操作的是哪个对象？

实例方法的第一个参数通常叫 self，表示当前实例。通过 a.area() 调用时，Python 自动把 a 传入该参数。

```
print(a.area())
print(Rectangle.area(a))
print(b.area())
```

实际输出

```
12
12
10
```

前两行在这里等价，都是读取 a 的宽和高；第三行读取 b 的。self 是约定的参数名，不是关键字。调用 a.area() 时不用再手动传 a。

对应 Notebook：8.5

__init__：把输入保存到实例中

以 a = Rectangle(3, 4) 为例：实例创建后，__init__ 接收到 self、width=3、height=4。

self.width = width：右侧 width 是本次调用的参数；左侧 self.width 是保存在实例上的属性。初始化结束后，仍能通过 a.width 访问它。

__init__ 是初始化方法，不负责返回实例；不要在其中返回业务结果。默认对象创建机制还涉及 __new__，本节不展开。

对应 Notebook：8.6

属性与方法：什么时候加括号？

读取属性用点号；执行方法用点号和圆括号。方法本身也是一个可被引用的对象。

```
print(a.width)
print(a.area())
area_function = a.area
print(callable(area_function))
print(area_function())
```

实际输出

```
3
12
True
12
```

a.area 取得绑定到 a 的方法，a.area() 才执行它。a.width 保存整数，写成 a.width() 会因整数不可调用而报 TypeError。

对应 Notebook：8.7

area() 返回一个计算结果，不修改矩形。直接给属性赋值会改变对象的状态；下一次计算使用新状态。

```
a.width = 6
result = a.area()
print(result)
print(b.area())
```

实际输出

```
24
10
```

a 的面积变成 $6 \times 4 = 24$；b 仍是 10。return 使结果可以继续参与运算；只有 print 而没有 return 的方法默认返回 None。

对应 Notebook：8.8

你已经用过对象和方法

列表和字符串也是对象。可以先学会使用某个类的实例，再逐步学习怎样自定义类。

```
numbers = [3, 1, 2]
returned = numbers.append(4)
print(numbers, returned)
name = " python "
print(name.strip().upper())
print(repr(name))
```

实际输出

```
[3, 1, 2, 4] None
PYTHON
' python '
```

append 修改列表并返回 None；strip 和 upper 返回新字符串，原字符串仍有空格。判断一个方法的行为，要看其约定，不能仅凭圆括号猜测。

对应 Notebook：8.9

两个变量可以指向同一实例。先预测：改 alias.width 后，original 的面积会不会变？

```
original = Rectangle(3, 4)
alias = original
independent = Rectangle(3, 4)
alias.width = 8
print(original.area(), independent.area())
print(alias is original)
print(independent is original)
```

实际输出

```
32 12
True
False
```

original 和 alias 指向同一个对象，面积为 32；independent 是新建对象，面积仍为 12。is 比较是否为同一对象。

对应 Notebook：8.10

问题：一笔持仓现在值多少钱？

现在把矩形的“数据 + 操作”迁移到持仓。一个 StockPosition 实例表示一只股票的一笔持仓。全部数字均为自编教学数据，不需要下载行情。

symbol 是代码字符串；shares 是股数（股）；cost_price 是每股买入成本；price 是当前每股价格，价格单位均为元/股。

手算：100 股，成本 10 元，现价 12 元。市值为 $100 \times 12 = 1200$ 元，未实现盈亏为 $100 \times (12 - 10) = 200$ 元。暂不计费用、分红及交易。

对应 Notebook：8.11

持仓类：保存字段，定义两个计算方法

每个属性只保存基础信息；市值与盈亏在调用时由当前价格计算，避免另存一份容易过期的结果。

```python
class StockPosition:
    def __init__(self, symbol, shares, cost_price, price):
        self.symbol = symbol
        self.shares = shares
        self.cost_price = cost_price
        self.price = price

    def market_value(self):
        return self.shares * self.price

    def unrealized_pnl(self):
        return self.shares * (self.price - self.cost_price)

    def update_price(self, new_price):
        self.price = new_price
```

market_value 和 unrealized_pnl 返回金额；update_price 修改状态，没有显式 return，因此返回 None。教学示例假定股数与价格为非负数。这也是封装：将持仓数据和相关操作放在同一个类中，提供统一的调用接口；本例属性仍可直接访问。

对应 Notebook：8.12

代码是字符串，避免丢掉前导零。下例只表示一笔持仓，不代表完整账户或真实成交。

```
position = StockPosition("000001", 100, 10.0, 12.0)
print(position.symbol, position.shares)
print(f"Market value: {position.market_value():.2f}")
print(f"Unrealized P&L: {position.unrealized_pnl():.2f}")
```

实际输出

```
000001 100
Market value: 1200.00
Unrealized P&L: 200.00
```

第一行是代码和股数；随后是市值 1200 元、未实现盈亏 200 元。“未实现”表示尚未卖出；这些数值只是所给价格下的估值。

对应 Notebook：8.13

将价格改为 11 元/股。先预测：股数、成本、市值和盈亏各有哪些变化？

```
returned = position.update_price(11.0)
print(returned)
print(position.shares, position.cost_price)
print(position.market_value(), position.unrealized_pnl())
```

实际输出

```
None
100 10.0
1100.0 100.0
```

股数仍为 100，成本仍为 10；市值变成 1100 元，盈亏变成 100 元。不要写 position = position.update_price(11.0)，否则变量会被重新绑定到 None。

对应 Notebook：8.14

对象列表：用同一种方法处理多笔持仓

列表的每个元素是一笔持仓对象，循环中 p 指向当前元素。本例重新创建数据，避免依赖前面改过的价格。

```
positions = [
    StockPosition("000001", 100, 10.0, 12.0),
    StockPosition("600000", 200, 8.0, 7.5),
]
total_value = 0
total_pnl = 0
for p in positions:
    print(p.symbol, p.market_value(), p.unrealized_pnl())
    total_value += p.market_value()
    total_pnl += p.unrealized_pnl()
print(total_value, total_pnl)
```

实际输出

```
000001 1200.0 200.0
600000 1500.0 -100.0
2700.0 100.0
```

两笔市值分别为 1200、1500 元；盈亏为 200、−100 元。合计市值 2700 元、盈亏 100 元。这里只相加相同币种的金额。

对应 Notebook：8.15

Square 是一种特殊的 Rectangle。通过继承复用 area，通过 super() 调用父类初始化，避免重复写面积公式。

```
class Square(Rectangle):
    def __init__(self, side):
        super().__init__(side, side)

square = Square(3)
print(square.area())
print(isinstance(square, Rectangle))
```

实际输出

```
9
True
```

边长 3，面积为 9。这个简化实现没有阻止分别修改宽和高，不能保证修改后仍为正方形；本例只演示继承与初始化。

对应 Notebook：8.16

重写方法：同名操作可以有不同实现（选学）

子类可以重新定义父类已有的方法。调用哪个实现，取决于当前对象的类。

```
class Greeter:
    def greet(self):
        return "Hello"

class ChineseGreeter(Greeter):
    def greet(self):
        return "你好"

for speaker in [Greeter(), ChineseGreeter()]:
    print(speaker.greet())
```

实际输出

```
Hello
你好
```

循环使用相同的 greet() 调用，得到不同问候语。这是多态的一个例子：同一个接口由不同对象实现。

对应 Notebook：8.17

多态也可以不依赖共同父类（选学）

Python 常按对象能否提供所需方法来使用它。下面两个类没有自定义的共同父类，但都有 market_value()。

```
class Cash:
    def __init__(self, amount):
        self.amount = amount

    def market_value(self):
        return self.amount

assets = [StockPosition("000001", 100, 10.0, 12.0),
          Cash(800.0)]
print(sum(asset.market_value() for asset in assets))
```

实际输出

```
2000.0
```

合计 2000 元。调用者关心的是能否按同一含义返回以元计的市值；这种使用方式通常称为鸭子类型，方法名相同还必须有一致的业务含义。

对应 Notebook：8.18

Portfolio 不是一种 StockPosition，而是包含多个资产对象。用组合关系组织“有一个/有多个”。

```python
class Portfolio:
    def __init__(self, assets):
        self.assets = list(assets)

    def market_value(self):
        return sum(a.market_value() for a in self.assets)

portfolio = Portfolio(assets)
print(portfolio.market_value())
assets[0].update_price(11.0)
print(portfolio.market_value())
```

实际输出

```
2000.0
1900.0
```

市值从 2000 变成 1900 元。list(assets) 复制了外层列表，但内部仍是相同资产对象，所以更新股票价格会影响组合估值。

对应 Notebook：8.19

什么时候需要自己写类?

函数：一次明确的输入到输出，例如计算收益率。

类：同一份状态需要多种操作，或需要创建许多有相同行为的实例，例如更新和估值多笔持仓。

字典：少量字段记录也很直观，不必为了面向对象而写类。后续 NumPy、pandas 已提供成熟对象；批量表格运算仍优先学习数组和表格操作。

封装组织接口；继承复用和扩展；多态让统一接口处理不同对象；组合把多个对象组装起来。本节不展开多重继承、元类和复杂设计模式。

对应 Notebook：8.20

以下是需要手动尝试的错误写法，主 Notebook 不会执行这些错误。

Rectangle(3)：缺少 height 参数，出现 TypeError。

a.heigth：拼错属性名，出现 AttributeError。

a.width()：将整数当函数调用，出现 TypeError。

方法中写 width 而非 self.width：不会自动读取实例属性；没有同名可用变量时出现 NameError。

先读报错最后一行，再定位代码行，核对对象、属性名、参数和括号。

对应 Notebook：8.21

练习 1：补全矩形的周长方法

参照 Rectangle，定义 PracticeRectangle，包含初始化、area() 和 perimeter()。宽 3、高 4 时，面积和周长各是多少？再建立宽 5、高 2 的第二个实例。先手算与画对象关系，再打开 Notebook 编写代码。

参考答案放在本节练习之后，讨论后再翻页。



练习 2：预测引用与修改

建立 first = Rectangle(2, 5)，执行 second = first、second.height = 8。
first.area() 是多少？怎样再创建一个不受此次修改影响的 2×5 矩形？先手算与画对象关系，再打开 Notebook 编写代码。
参考答案放在本节练习之后，讨论后再翻页。



练习 3：持仓汇总与价格变化

创建 A：100 股、成本 10、现价 12；B：200 股、成本 8、现价 7.5。计算合计市值和盈亏，再将 A 价格更新为 11 并重新汇总。先手算，后写循环。先手算与画对象关系，再打开 Notebook 编写代码。

参考答案放在本节练习之后，讨论后再翻页。



创建 100 股、现价 12 的持仓与 800 元现金，放入 Portfolio。若股票涨至 13，组合价值是多少？解释为何 Portfolio 不应继承 StockPosition。先手算与画对象关系，再打开 Notebook 编写代码。

参考答案放在本节练习之后，讨论后再翻页。



class 定义类；调用类创建实例；__init__ 初始化实例属性。
self 指当前实例；点号读取属性；圆括号执行方法。
赋值可以让多个名字引用同一个对象；需要独立状态时明确创建新对象。
持仓保存代码、股数、成本和现价；方法更新价格并计算市值、未实现盈亏。
回到本章综合案例，练习选择合适的数据结构与函数；下一章继续识别 NumPy 对象的属性与方法。



参考答案 1：补全矩形的周长方法

```python
class PracticeRectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

r1 = PracticeRectangle(3, 4)
r2 = PracticeRectangle(5, 2)
print(r1.area(), r1.perimeter())
print(r2.area(), r2.perimeter())
```

分别为面积 12、周长 14；面积 10、周长 14。面积相同与周长相同是不同的问题。

参考答案 2：预测引用与修改

```
first = Rectangle(2, 5)
second = first
separate = Rectangle(2, 5)
second.height = 8
print(first.area(), separate.area())
print(first is second, first is separate)
```

输出面积 16 与 10；first 与 second 是同一个对象，separate 是独立新建的实例。

参考答案 3：持仓汇总与价格变化

```python
holdings = [StockPosition("A", 100, 10.0, 12.0),
            StockPosition("B", 200, 8.0, 7.5)]
def totals(holdings):
    value = 0
    pnl = 0
    for holding in holdings:
        value += holding.market_value()
        pnl += holding.unrealized_pnl()
    return value, pnl

print(totals(holdings))
holdings[0].update_price(11.0)
print(totals(holdings))
```

初始市值 2700 元、盈亏 100 元；更新后市值 2600 元、盈亏 0 元。股数与成本没有变化。

参考答案 4（选学）：为估值接口增加现金

```python
stock = StockPosition("A", 100, 10.0, 12.0)
cash = Cash(800.0)
account = Portfolio([stock, cash])
stock.update_price(13.0)
print(account.market_value())
```

价值为 2100 元。Portfolio 包含持仓与现金，因此用组合；它本身不是一笔股票持仓。

1. 初识 Python 程序
2. 基本数据类型
3. 运算符与表达式
4. 字符串与格式化输出
5. 基本数据结构
6. 控制流：让程序会” 判断” 与” 重复”
7. 函数：把常用计算” 打包”
8. 类与对象：面向对象入门
9. 综合案例与小结

```python
# 某股票近5日收盘价
prices = [100.0, 102.0, 101.0, 105.0, 103.0]

# 1. 区间收益率
total_ret = (prices[-1] - prices[0]) / prices[0]

# 2. 每日收益率，并统计上涨天数
up_days = 0
for i in range(1, len(prices)):
    r = (prices[i] - prices[i-1]) / prices[i-1]
    if r > 0:
        up_days += 1

print(f"区间收益率: {total_ret:.2%}")
print(f"上涨天数: {up_days} / {len(prices)-1}")
```

本章学的**变量、列表、循环、判断、f-string** 在这里全部用上了。

- **变量与赋值**：`name = value`；= 是赋值不是相等。
- **基本类型**：`int`、`float`、`str`、`bool`；用 `type()` 检查，用类型名转换。
- **运算符**：`/ // % **`；比较用 `==`；逻辑用 `and or not`。
- **字符串**：索引从 0 开始、切片含头不含尾；`f"{x:.2%}"` 格式化。
- **数据结构**：列表（可变序列）、元组（不可变）、字典（键值对）、集合（去重）。
- **控制流**：`if/elif/else`、`for`、`while`；缩进即语法。
- **函数**：`def` 定义，`return` 返回，复用与封装。
- **类与对象**：`class` 定义类，实例保存状态，方法提供操作。

课后任务

1. 在 Notebook 中完成本章所有课堂练习并截图保存。
2. 编写脚本：输入本金、年利率、年限，打印复利终值。
3. 完成矩形、对象引用与持仓估值练习；选做组合估值。
4. 选做：用列表与循环计算一组价格的最大值与最小值（不许用 `max/min`）。

下章预告

本章逐个元素地循环计算，处理上万行行情数据会很慢。
下一章学习 **NumPy 数组**：一行代码对整个数组做运算，速度提升百倍。

本章内容改编自教材第 3 章[1]。

扩展阅读与练习来源

- Wes McKinney, *Python for Data Analysis*, 3rd ed.
- `https://wesmckinney.com/book/python-builtin`
- 新增中文说明、合成数据与练习为本课程自行编写。
- 阅读正文理解概念；在配套 Notebook 中运行、修改、核对。

[1] 伊夫·希尔皮斯科. Python 金融大数据分析 [M]. 2nd ed. 北京 : 人民邮电出版社, 2018.



Thanks for your attention!

Q & A